Project Day 5

Name: Kaif Ajaz Shaikh

Roll No.:30

Branch: Computer

College : Theem College Of Engineering

Project 1:Simple Interest calculator

Code:

```
from tkinter import *
from tkinter import messagebox


def calculate_interest():
    try:
        # Retrieve values from entry fields
        principal = float(entry_principal.get())
        rate = float(entry_rate.get())
        time = float(entry_time.get())

        # Simple Interest formula
        interest = (principal * rate * time) / 100
        total_amount = principal + interest

        # Update the result labels
        label_interest_val.config(text=f"{interest:.2f}")
        label_total_val.config(text=f"{total_amount:.2f}")

    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter valid numerical values.")

def clear_fields():
    # Clear all input entries and reset results
    entry_principal.delete(0, END)
    entry_rate.delete(0, END)
    entry_time.delete(0, END)
    label_interest_val.config(text="0.00")
```

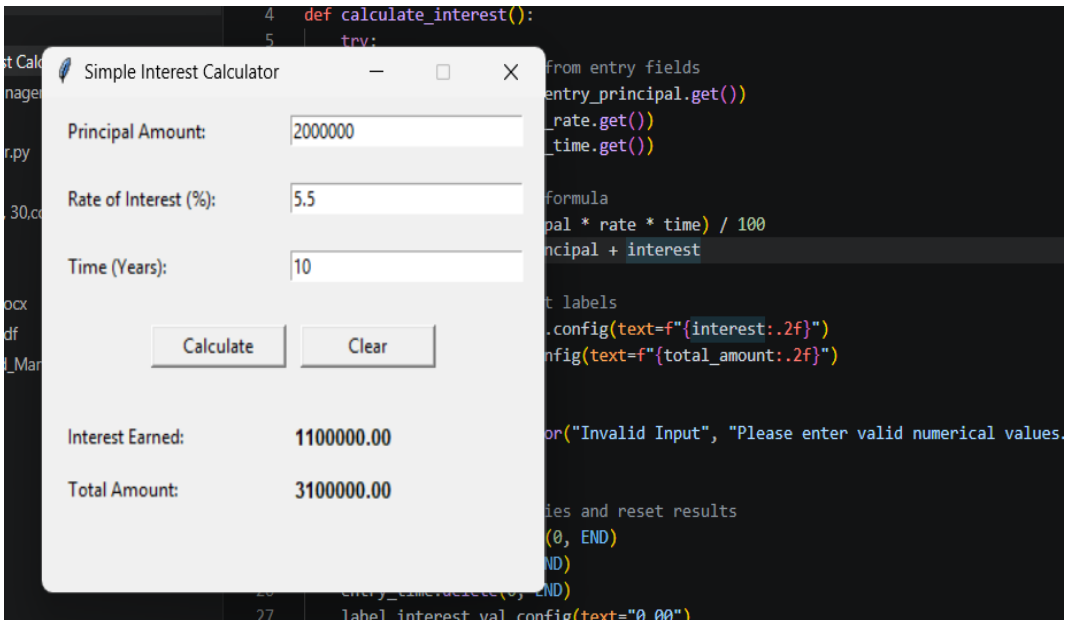
```
    label_total_val.config(text="0.00")


# Main Application Window
root = Tk()
root.title("Simple Interest Calculator")
root.geometry("350x300")
root.resizable(False, False)

# Main Grid Layout Configuration
root.columnconfigure(0, weight=1)
root.columnconfigure(1, weight=2)

# --- Input Fields ---
Label(root, text="Principal Amount:").grid(row=0, column=0, padx=15, pady=10, sticky="w")
entry_principal = Entry(root)
entry_principal.grid(row=0, column=1, padx=15, pady=10, sticky="ew")

Label(root, text="Rate of Interest (%):").grid(row=1, column=0, padx=15, pady=10, sticky="w")
entry_rate = Entry(root)
entry_rate.grid(row=1, column=1, padx=15, pady=10, sticky="ew")

Label(root, text="Time (Years):").grid(row=2, column=0, padx=15, pady=10, sticky="w")
entry_time = Entry(root)
entry_time.grid(row=2, column=1, padx=15, pady=10, sticky="ew")

# --- Buttons ---
frame_buttons = Frame(root)
frame_buttons.grid(row=3, column=0, columnspan=2, pady=15)
btn_calculate = Button(frame_buttons, text="Calculate", command=calculate_interest, width=12)
btn_calculate.pack(side="left", padx=5)
```

```
btn_clear = Button(frame_buttons, text="Clear", command=clear_fields, width=12)

btn_clear.pack(side="left", padx=5)

# --- Separation Line ---

Frame(root, height=1, bd=1, relief="sunken").grid(row=4, column=0, columnspan=2, sticky="ew", padx=15, pady=5)

# --- Output Results ---

Label(root, text="Interest Earned:").grid(row=5, column=0, padx=15, pady=5, sticky="w")

label_interest_val = Label(root, text="0.00", font=("Arial", 10, "bold"))

label_interest_val.grid(row=5, column=1, padx=15, pady=5, sticky="w")

Label(root, text="Total Amount:").grid(row=6, column=0, padx=15, pady=5, sticky="w")

label_total_val = Label(root, text="0.00", font=("Arial", 10, "bold"))

label_total_val.grid(row=6, column=1, padx=15, pady=5, sticky="w")

# Start the application loop

root.mainloop()
```

Simple Interest Calculator.py | Banking System.py | EMPData.py | RailONE.py

DAY 5 > Simple Interest Calculator.py > calculate_interest

```python
from tkinter import *
from tkinter import messagebox

def calculate_interest():
    try:
        # Retrieve values from entry fields
        principal = float(entry_principal.get())
        rate = float(entry_rate.get())
        time = float(entry_time.get())

        # Simple Interest formula
        interest = (principal * rate * time) / 100
        total_amount = principal + interest

        # Update the result labels
        label_interest_val.config(text=f"{interest:.2f}")
        label_total_val.config(text=f"{total_amount:.2f}")

    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter valid numerical values.")

def clear_fields():
    # Clear all input entries and reset results
    entry_principal.delete(0, END)
    entry_rate.delete(0, END)
    entry_time.delete(0, END)
    label_interest_val.config(text="0.00")
    label_total_val.config(text="0.00")
```



Project 2:Bank System GUI

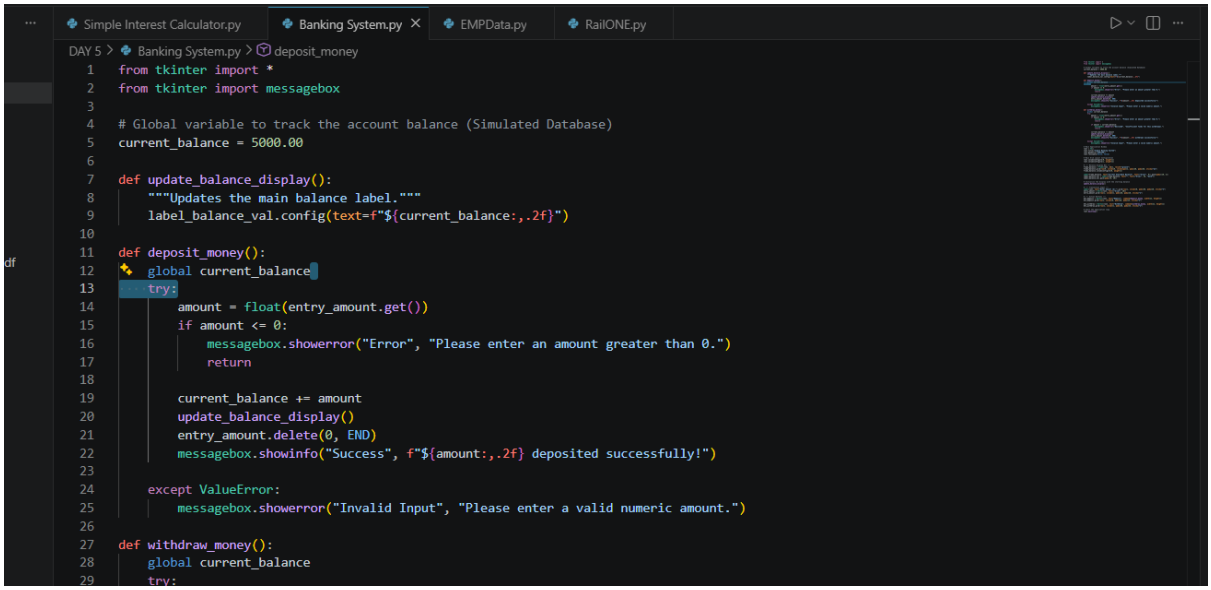
Code:

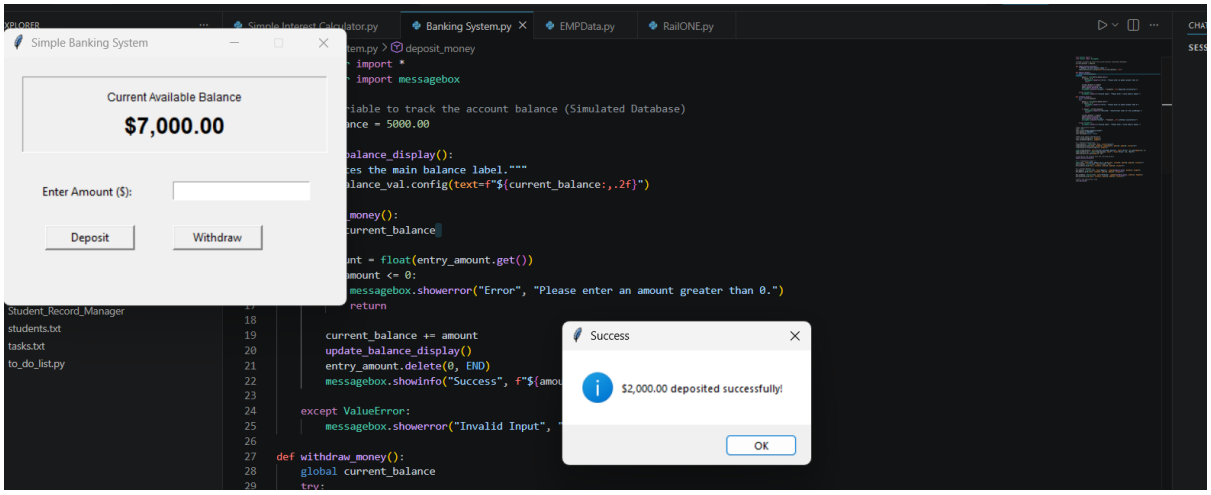
```
from tkinter import *
from tkinter import messagebox

# Global variable to track the account balance (Simulated Database)
current_balance = 5000.00

def update_balance_display():
    """Updates the main balance label."""
    label_balance_val.config(text=f"${current_balance:,.2f}")

def deposit_money():
    global current_balance
    try:
        amount = float(entry_amount.get())
        if amount <= 0:
            messagebox.showerror("Error", "Please enter an amount greater than 0.")
            return

        current_balance += amount
        update_balance_display()
        entry_amount.delete(0, END)
        messagebox.showinfo("Success", f"${amount:,.2f} deposited successfully!")

    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter a valid numeric amount.")
```

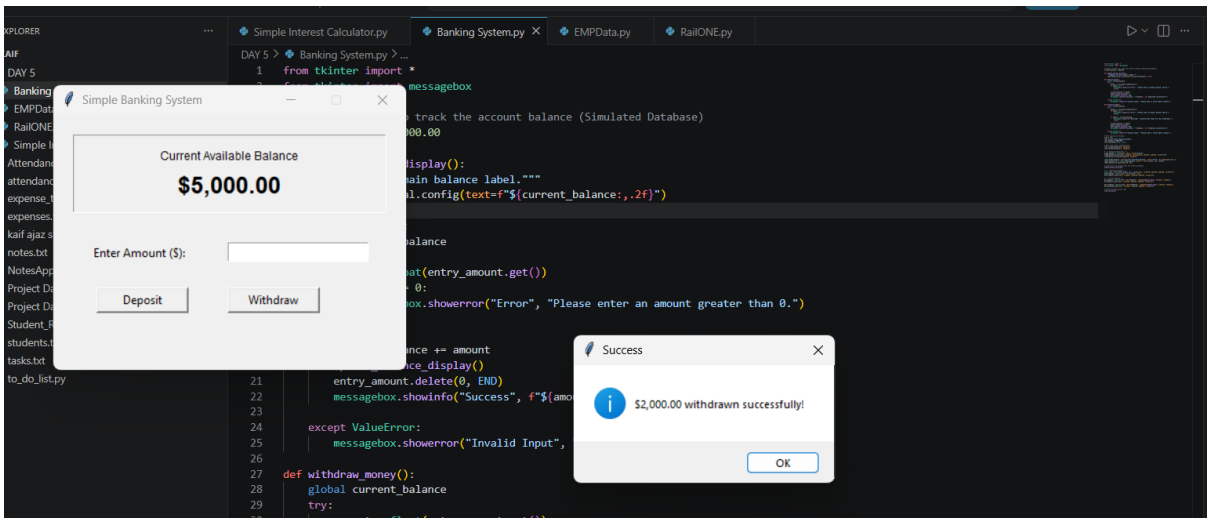
```
def withdraw_money():
    global current_balance
    try:
        amount = float(entry_amount.get())
        if amount <= 0:
            messagebox.showerror("Error", "Please enter an amount greater than 0.")
            return

        if amount > current_balance:
            messagebox.showerror("Declined", "Insufficient funds for this withdrawal.")
            return
        current_balance -= amount
        update_balance_display()
        entry_amount.delete(0, END)
        messagebox.showinfo("Success", f"${amount:,.2f} withdrawn successfully!")

    except ValueError:
        messagebox.showerror("Invalid Input", "Please enter a valid numeric amount.")

# Main Application Window
root = Tk()
root.title("Simple Banking System")
root.geometry("360x260")
root.resizable(False, False)

# Main Grid Layout Configuration
root.columnconfigure(0, weight=1)
```

```
root.columnconfigure(1, weight=1)

# --- Balance Display Box ---
frame_balance = Frame(root, bd=1, relief="groove")
frame_balance.grid(row=0, column=0, columnspan=2, padx=20, pady=20, sticky="ew")
frame_balance.columnconfigure(0, weight=1)

Label(frame_balance, text="Current Available Balance", font=("Arial", 9)).pack(pady=(10, 2))
label_balance_val = Label(frame_balance, text="", font=("Arial", 18, "bold"))
label_balance_val.pack(pady=(0, 10))

# Initialize the display with the starting balance
update_balance_display()

# --- Transaction Input ---
Label(root, text="Enter Amount ($):").grid(row=1, column=0, padx=20, pady=10, sticky="e")
entry_amount = Entry(root, font=("Arial", 10))
entry_amount.grid(row=1, column=1, padx=20, pady=10, sticky="w")

# --- Action Buttons ---
btn_deposit = Button(root, text="Deposit", command=deposit_money, width=12, height=1)
btn_deposit.grid(row=2, column=0, padx=20, pady=15, sticky="e")

btn_withdraw = Button(root, text="Withdraw", command=withdraw_money, width=12,
height=1)
btn_withdraw.grid(row=2, column=1, padx=20, pady=15, sticky="w")
# Start the application loop
root.mainloop()
```

2.CODE And O/P


Simple Interest Calculator.py
Banking System.py
EMPData.py
RailONE.py
DAY 5 > Banking System.py > deposit_money
from tkinter import *
from tkinter import messagebox
# Global variable to track the account balance (Simulated Database)
current_balance = 5000.00
def update_balance_display():
"""Updates the main balance label."""
label_balance_val.config(text=f"${current_balance:,.2f}")
def deposit_money():
global current_balance
try:
amount = float(entry_amount.get())
if amount <= 0:
messagebox.showerror("Error", "Please enter an amount greater than 0.")
return
current_balance += amount
update_balance_display()
entry_amount.delete(0, END)
messagebox.showinfo("Success", f"${amount:,.2f} deposited successfully!")
except ValueError:
messagebox.showerror("Invalid Input", "Please enter a valid numeric amount.")
def withdraw_money():
global current_balance
try:


Simple Banking System
Current Available Balance
$7,000.00
Enter Amount ($):
Deposit
Withdraw
Success
$2,000.00 deposited successfully!
OK


Simple Banking System
Current Available Balance
$5,000.00
Enter Amount ($):
Deposit
Withdraw
Success
$2,000.00 withdrawn successfully!
OK

Project 3:Employee Database GUI

Code:

```
from tkinter import *
from tkinter import messagebox


# Global list to hold employee dictionaries (Simulated Database)
employee_db = [
    {"id": "101", "name": "Alice Smith", "role": "Manager"},
    {"id": "102", "name": "Bob Jones", "role": "Developer"}
]


def refresh_employee_list():
    """Clears and repopulates the listbox with current data."""
    listbox_employees.delete(0, END)
    for emp in employee_db:
        # Format string for display in the listbox
        display_text = f"ID: {emp['id']} | {emp['name']} — ({emp['role']})"
        listbox_employees.insert(END, display_text)


def add_employee():
    emp_id = entry_id.get().strip()
    name = entry_name.get().strip()
    role = entry_role.get().strip()

    # Validation checks
    if not emp_id or not name or not role:
        messagebox.showerror("Error", "All fields are required.")
        return

    # Check if Employee ID already exists
    for emp in employee_db:
```

```
        if emp['id'] == emp_id:
            messagebox.showerror("Error", f"Employee ID {emp_id} already exists.")
            return

    # Add to our local list database
    employee_db.append({"id": emp_id, "name": name, "role": role})

    # Refresh UI and clear entries
    refresh_employee_list()
    entry_id.delete(0, END)
    entry_name.delete(0, END)
    entry_role.delete(0, END)
    messagebox.showinfo("Success", f"Employee '{name}' added successfully.")

def delete_employee():
    try:
        # Get the selected index from the listbox
        selected_index = listbox_employees.curselection()[0]

        # Remove it from our list database
        removed_emp = employee_db.pop(selected_index)

        # Refresh UI
        refresh_employee_list()
        messagebox.showinfo("Deleted", f"Removed {removed_emp['name']} from the system.")

    except IndexError:
        messagebox.showerror("Error", "Please select an employee from the list to delete.")

# Main Application Window
root = Tk()
```

```
root.title("Employee Database System")
root.geometry("480x400")
root.resizable(False, False)
# --- Left Side: Input Form ---
frame_form = LabelFrame(root, text=" Add New Employee ", padx=10, pady=10)
frame_form.place(x=15, y=15, width=200, height=310)

Label(frame_form, text="Employee ID:").pack(anchor="w", pady=(5, 2))
entry_id = Entry(frame_form)
entry_id.pack(fill="x", pady=(0, 10))

Label(frame_form, text="Full Name:").pack(anchor="w", pady=(0, 2))
entry_name = Entry(frame_form)
entry_name.pack(fill="x", pady=(0, 10))

Label(frame_form, text="Role/Position:").pack(anchor="w", pady=(0, 2))
entry_role = Entry(frame_form)
entry_role.pack(fill="x", pady=(0, 15))

btn_add = Button(frame_form, text="Add Employee", command=add_employee, height=1)
btn_add.pack(fill="x")
# --- Right Side: Database Records View ---
frame_records = LabelFrame(root, text=" Current Records ", padx=10, pady=10)
frame_records.place(x=230, y=15, width=235, height=310)

# Scrollbar for the records list box
scrollbar = Scrollbar(frame_records, orient="vertical")
listbox_employees = Listbox(frame_records, yscrollcommand=scrollbar.set, exportselection=False)
scrollbar.config(command=listbox_employees.yview)

scrollbar.pack(side="right", fill="y")
```

listbox_employees.pack(side="left", fill="both", expand=True)

--- Bottom Action Frame ---

btn_delete = Button(root, text="Delete Selected Employee", command=delete_employee, width=22)

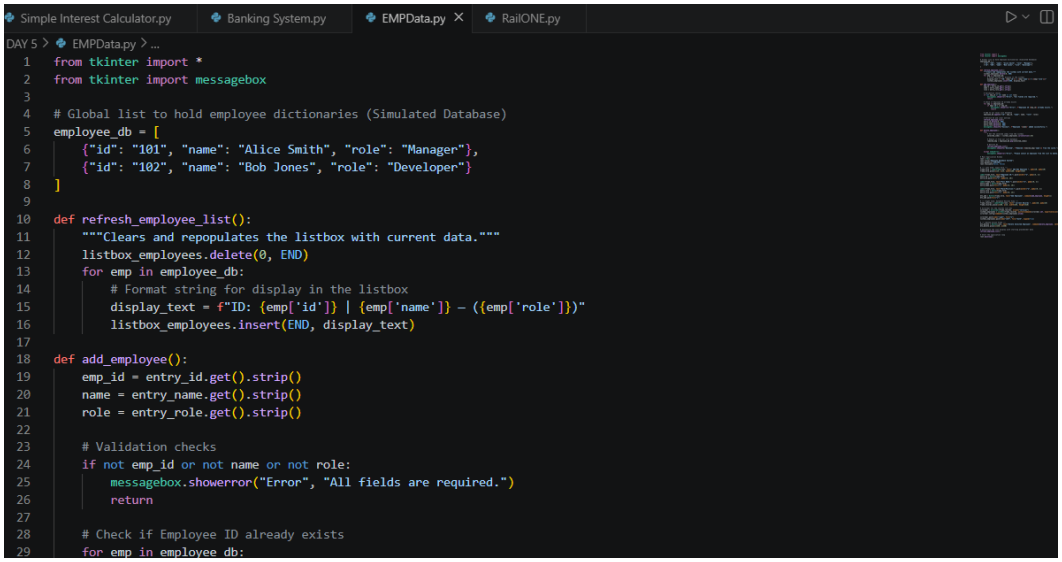
btn_delete.place(x=230, y=340)

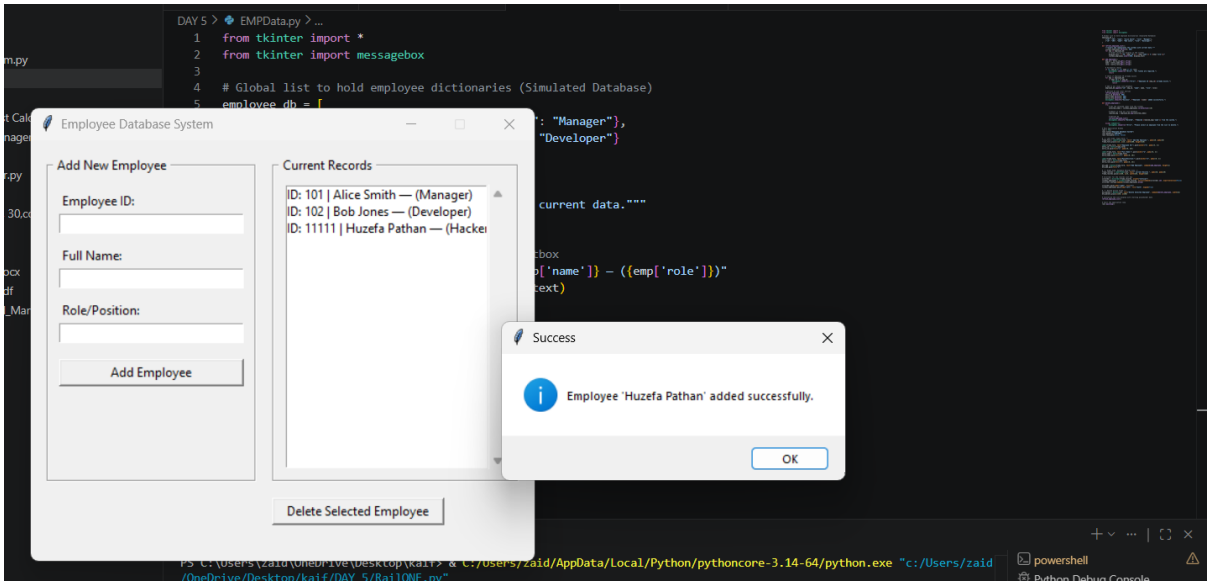
Initialize the list display with starting placeholder data

refresh_employee_list()

Start the application loop

root.mainloop()





Project 4:RailONe App GUI

Code:

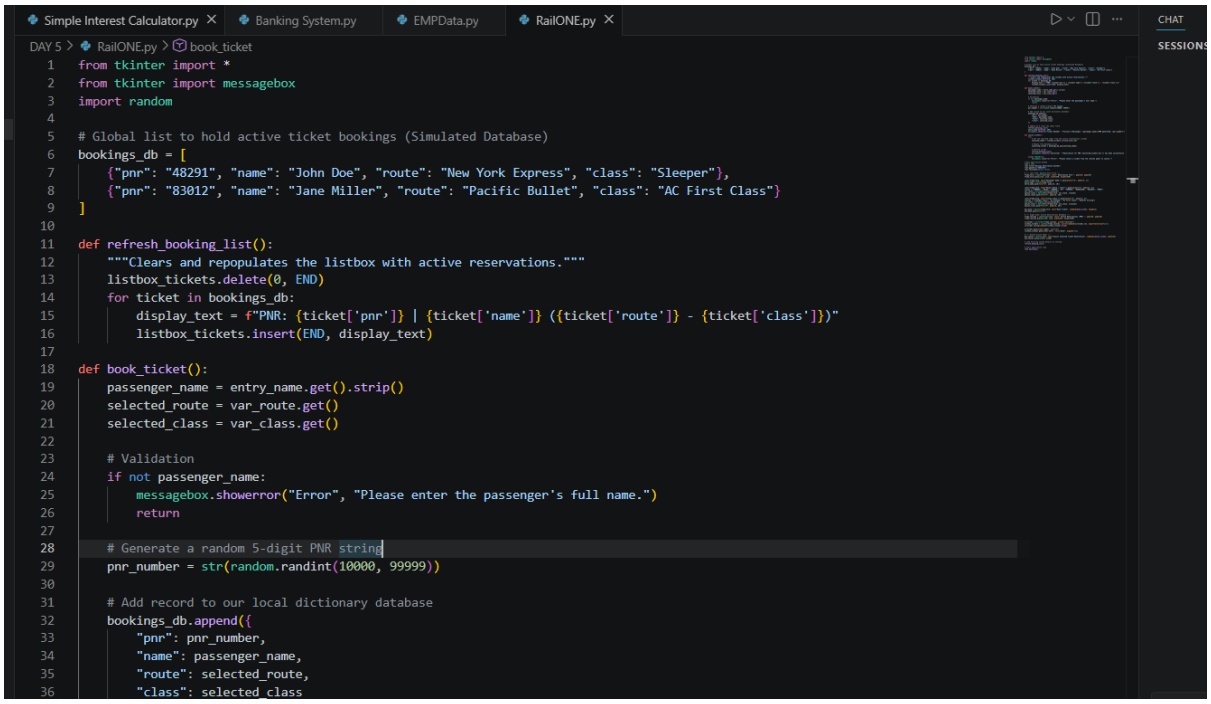
```
from tkinter import *
from tkinter import messagebox
import random

# Global list to hold active ticket bookings (Simulated Database)
bookings_db = [
    {"pnr": "48291", "name": "John Doe", "route": "New York Express", "class": "Sleeper"},
    {"pnr": "83012", "name": "Jane Miller", "route": "Pacific Bullet", "class": "AC First Class"}
]

def refresh_booking_list():
    """Clears and repopulates the listbox with active reservations."""
    listbox_tickets.delete(0, END)
    for ticket in bookings_db:
        display_text = f"PNR: {ticket['pnr']} | {ticket['name']} ({ticket['route']} - {ticket['class']})"
        listbox_tickets.insert(END, display_text)

def book_ticket():
    passenger_name = entry_name.get().strip()
    selected_route = var_route.get()
    selected_class = var_class.get()

    # Validation
    if not passenger_name:
        messagebox.showerror("Error", "Please enter the passenger's full name.")
```

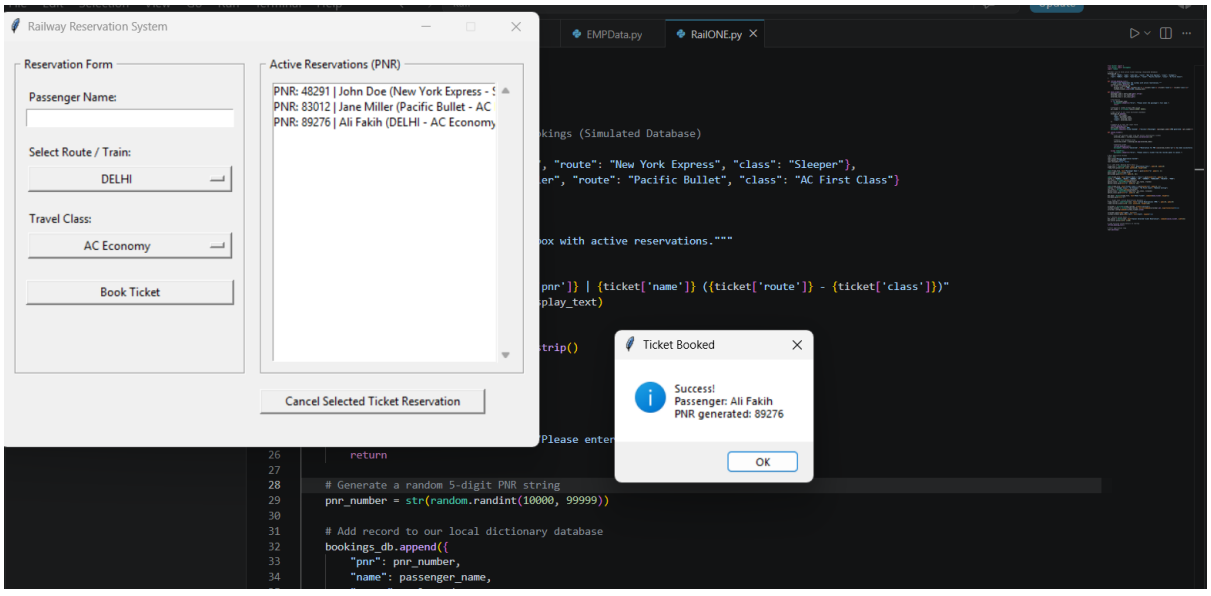
```
        return

    # Generate a random 5-digit PNR string
    pnr_number = str(random.randint(10000, 99999))

    # Add record to our local dictionary database
    bookings_db.append({
        "pnr": pnr_number,
        "name": passenger_name,
        "route": selected_route,
        "class": selected_class
    })

    # Update UI & clear the input field
    refresh_booking_list()
    entry_name.delete(0, END)
    messagebox.showinfo("Ticket Booked", f"Success!\nPassenger: {passenger_name}\nPNR generated: {pnr_number}")

def cancel_ticket():
    try:
        # Get the selected index from the active reservations listbox
        selected_index = listbox_tickets.curselection()[0]

        # Remove from database array
        cancelled_ticket = bookings_db.pop(selected_index)

        # Refresh screen
```

```
        refresh_booking_list()
        messagebox.showinfo("Cancelled", f"Reservation for PNR {cancelled_ticket['pnr']} has been successfully cancelled.")

    except IndexError:
        messagebox.showerror("Error", "Please select a ticket from the records panel to cancel.")

# Main Application Window
root = Tk()
root.title("Railway Reservation System")
root.geometry("560x420")
root.resizable(False, False)

# --- Left Side: Booking Entry Form ---
frame_form = LabelFrame(root, text=" Reservation Form ", padx=10, pady=10)
frame_form.place(x=15, y=15, width=240, height=330)

Label(frame_form, text="Passenger Name:").pack(anchor="w", pady=(5, 2))
entry_name = Entry(frame_form)
entry_name.pack(fill="x", pady=(0, 10))

Label(frame_form, text="Select Route / Train:").pack(anchor="w", pady=(5, 2))
routes = ["MUMBAI", "DELHI", "HOWRAH", "UP", "CHENNAI", "BANGALORE", "KOLKATA", "PUNE"]
var_route = StringVar(value=routes[0])
option_route = OptionMenu(frame_form, var_route, *routes)
option_route.pack(fill="x", pady=(0, 10))
```

```
Label(frame_form, text="Travel Class:").pack(anchor="w", pady=(5, 2))

classes = ["Sleeper Class", "AC Economy", "AC First Class", "General Sitting"]

var_class = StringVar(value=classes[0])

option_class = OptionMenu(frame_form, var_class, *classes)

option_class.pack(fill="x", pady=(0, 20))


btn_book = Button(frame_form, text="Book Ticket", command=book_ticket, height=1)

btn_book.pack(fill="x")


# --- Right Side: Active Reservations Database ---

frame_records = LabelFrame(root, text=" Active Reservations (PNR) ", padx=10, pady=10)

frame_records.place(x=270, y=15, width=275, height=330)


scrollbar = Scrollbar(frame_records, orient="vertical")

listbox_tickets = Listbox(frame_records, yscrollcommand=scrollbar.set,
exportselection=False)

scrollbar.config(command=listbox_tickets.yview)


scrollbar.pack(side="right", fill="y")

listbox_tickets.pack(side="left", fill="both", expand=True)


# --- Bottom Action Panel ---

btn_cancel = Button(root, text="Cancel Selected Ticket Reservation",
command=cancel_ticket, width=32)

btn_cancel.place(x=270, y=360)

# Load existing system details on startup

refresh_booking_list()# Start application loop

root.mainloop()
```

Code &O/P


Simple Interest Calculator.py
Banking System.py
EMPData.py
RailONE.py
CHAT
SESSIONS
DAY 5 > RailONE.py > book_ticket
from tkinter import *
from tkinter import messagebox
import random
# Global list to hold active ticket bookings (Simulated Database)
bookings_db = [
{"pnr": "48291", "name": "John Doe", "route": "New York Express", "class": "Sleeper"},
{"pnr": "83012", "name": "Jane Miller", "route": "Pacific Bullet", "class": "AC First Class"}
]
def refresh_booking_list():
"""Clears and repopulates the listbox with active reservations."""
listbox_tickets.delete(0, END)
for ticket in bookings_db:
display_text = f"PNR: {ticket['pnr']} | {ticket['name']} ({ticket['route']} - {ticket['class']})"
listbox_tickets.insert(END, display_text)
def book_ticket():
passenger_name = entry_name.get().strip()
selected_route = var_route.get()
selected_class = var_class.get()
# Validation
if not passenger_name:
messagebox.showerror("Error", "Please enter the passenger's full name.")
return
# Generate a random 5-digit PNR string
pnr_number = str(random.randint(10000, 99999))
# Add record to our local dictionary database
bookings_db.append({
"pnr": pnr_number,
"name": passenger_name,
"route": selected_route,
"class": selected_class


Railway Reservation System
Reservation Form
Passenger Name:
Select Route / Train:
DELHI
Travel Class:
AC Economy
Book Ticket
Active Reservations (PNR)
PNR: 48291 | John Doe (New York Express -
PNR: 83012 | Jane Miller (Pacific Bullet - AC
PNR: 89276 | Ali Fakih (DELHI - AC Economy
Cancel Selected Ticket Reservation
Ticket Booked
Success!
Passenger: Ali Fakih
PNR generated: 89276
OK
EMPData.py
RailONE.py
return
# Generate a random 5-digit PNR string
pnr_number = str(random.randint(10000, 99999))
# Add record to our local dictionary database
bookings_db.append({
"pnr": pnr_number,
"name": passenger_name,